252-0027

Introduction to Programming

Exercises

Week 8: Classes, Loop Invariants

Jan Leonard
Department of Computer Science
ETH Zurich

Plan for today

- Organization
- Timed Bonus
- Classes - Theory & Example
- Kahoot
- Loop Invariants - Examples & Recipe
- Preview

Organization

- My name: *Jan Leonard*
- For questions: leonardj@ethz.ch
  - Please put «[EProg26]» in the subject
- New exercises: **Tuesday evening** (normally)
- Submit exercises by **Tuesday evening (23:59)** following week
  - Always submit via Git
  - Solutions in a separate Git project



Eprog TA — HS 2026

janleonard115.github.io



Mailing List

docs.google.com

Timed Bonus

- Timed Bonus is supposed to be hard
  - Do not let it get you down
- Please see it as a wake-up call
  - This is why we moved it to an earlier week
- There is still plenty of time left until the exam

- My first timed bonus went completely wrong

A valid input for the parameter `chars` is a two-dimensional `char` array of size $n \times m$ with $n, m \geq 1$.

Subtask 1: Invalid Inputs

Implement the method `getWord` such that the string `"Ungueltig"` is returned if the array passed with `chars` is not a valid input.

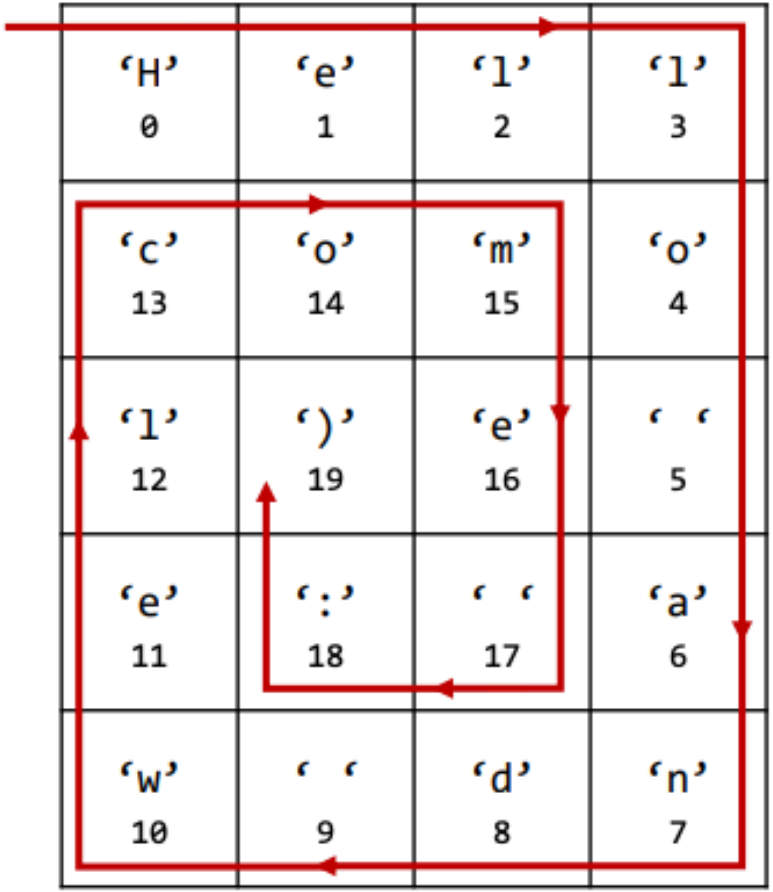
```
if (chars == null) {
    return "Ungueltig";
}
```

```
if (chars.length == 0) {
    return "Ungueltig";
}
```

```
if (chars[0] == null) {
    return "Ungueltig";
}
```

```
if (chars[0].length == 0) {
    return "Ungueltig";
}
```

```
int m = chars[0].length;
for (int i = 1; i < chars.length; i++) {
    if (chars[i] == null) {
        return "Ungueltig";
    }
    if (chars[i].length != m) {
        return "Ungueltig";
    }
}
```


‘H’ 0
‘e’ 1
‘l’ 2
‘l’ 3
‘c’ 13
‘o’ 14
‘m’ 15
‘o’ 4
‘l’ 12
‘)’ 19
‘e’ 16
‘ ‘ 5
‘e’ 11
‘:’ 18
‘ ‘ 17
‘a’ 6
‘w’ 10
‘ ‘ 9
‘d’ 8
‘n’ 7

```java
public static String spirale(char[][] chars, int top, int bottom, int left, int right)
    if (top > bottom || left > right) {
        return "";
    }

    String result = "";
    for (int j = left; j <= right; j++) {
        result += chars[top][j];
    }
    for (int i = top + 1; i <= bottom; i++) {
        result += chars[i][right];
    }
    if (top < bottom) {
        for (int j = right - 1; j >= left; j--) {
            result += chars[bottom][j];
        }
    }
    if (left < right) {
        for (int i = bottom - 1; i > top; i--) {
            result += chars[i][left];
        }
    }

    return result + spirale(chars, top + 1, bottom - 1, left + 1, right - 1);
}
```

Classes

Classes - Theory

- A class is a blueprint
  - Defines the data (attributes) and behavior (methods) of its objects
  - Object is an instance of a class

```
Point p = new Point(1, 2);
Point q = new Point(5, 7);
```

- Same Class - different Object

Classes - Theory

```java
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public int getX() {
        return x;
    }
}
```

Attributes

Constructors

Methods

Classes - Theory

- **Static**
  - Attributes or Methods declared as static belong to the class rather than the object
  - Static Methods are called on the class:
    - Point.getCount()
  - In static methods there is no current object

- Non-Static Methods are called instance methods

Classes - Theory

- **Constructors**
  - Have the **same name** as the class
  - No return type
  - Always called with Keyword **new**

```
public Point(int x, int y) {
    this.x = x;
    this.y = y;
}
```

```
Point p = new Point(3, 4);
```

- Signature of a constructor: **name and parameterlist**
  - Number, Type and order of Parameters

Classes - Theory

- **Constructors**
  - Default constructor
    - Invisible
    - Added automatically by Java if the class has no constructor
    - No parameters and does nothing
    - Disappears as soon as you write your own constructor

```
public class Point {
    private int x;
    private int y;
}
```

```
Point p = new Point();
```

Classes - Theory

- **Constructors**
  - Parameterless constructor
  - Parameterized constructor
  - Copy constructor

```
public Point() {
    this.x = 1;
    this.y = 1;
}
```

```
public Point(int x, int y) {
    this.x = x;
    this.y = y;
}
```

```
public Point(Point other) {
    this(other.x, other.y);
}
```

Classes - Theory

- **this**
  - **this** refers to the object the method is called on
  - If the method is not called on an object (static method) - ERROR
- We need **this** for shadowing inside constructors or methods

```
public Point(int x, int y) {
    this.x = x;
    this.y = y;
}
```

- We can use **this** whenever we need a reference to the current object

Classes - Theory

- **this()**
  - Is a call to a constructor inside a constructor (of the same class)
  - **this() does not create another object**
  - **this() can only be called inside another constructor**

```java
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public Point() {
        this(0, 0);
    }

    public Point(Point other) {
        this(other.x, other.y);
    }
}
```

Classes - Theory

- **Private/Public**
  - Defines how an attribute can be accessed
  - Private: From inside the **same class**
  - Public: From **everywhere**

- For private attributes we use Getter/Setter Methods
- (Constructors can be private too)

```java
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

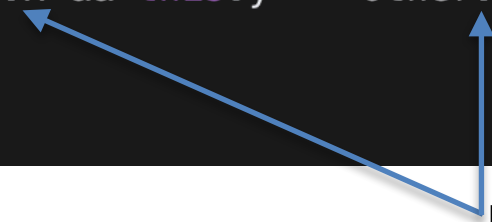
    public Point() {
        this(0, 0);
    }

    public Point(Point other) {
        this(other.x, other.y);
    }
}
```

```java
public class Point {
    private int x;
    private int y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public boolean isSameAs(Point other) {
        return this.x == other.x && this.y == other.y;
    }
}
```



Solving Problems: Zeitspanne

In this exercise, we want to create a class Zeitspanne (time span). Objects of this class store a duration (e.g. 3 hours, 45 minutes) and return it on request.

1. Create the class Zeitspanne. This class creates objects that represent a time span (in hours and minutes; seconds are not required).
2. Create suitable methods for this class. We should be able to change the time span (set it to any (sensible) value, add and subtract), return it in minutes, and print it.
3. Now write a method 'isMoreThan()' that takes a second Zeitspanne as an argument and makes it possible to compare two time spans.

```java
public class Zeitspanne {
    public int minutes;
    public int hours;
}
```

1. Create the class Zeitspanne. This class creates objects that represent a time span (in hours and minutes; seconds are not required).

```
public class Zeitspanne {
    public int minutes;
    public int hours;
}
```

Not necessary…

94 minutes = 1 hour and 34 minutes

1. Create the class Zeitspanne. This class creates objects that represent a time span (in hours and minutes; seconds are not required).

```
public class Zeitspanne {
    public int minutes;
}
```


Attributes should be private!

Access to attributes via getter / setter methods.

```java
public class Zeitspanne {
    private int minutes;

    // Getter Methoden
    public int getTimeInMin() {
        return this.minutes;
    }

    public int getFullHours() {
        return this.minutes / 60;
    }

}
```

A user would surely also want to know the number of
hours.

```java
public class Zeitspanne {
    private int minutes;

    // Getter Methoden
    public int getTimeInMin() {
        return this.minutes;
    }

    public int getFullHours() {
        return this.minutes / 60;
    }

}
```

```
(…)
    public int getFullHours() {
        return this.minutes / 60;
    }

    // Setter Methoden
    public void setMinutes(int minutes) {
        this.minutes = minutes;
    }
}
```

Should a user have (indirect) access to the `minutes` attribute?

Here: yes

But this is often not the case!

2. Create suitable methods for this class. We should be able to change the time span (set it to any (sensible) value, add and subtract), return it in minutes, and print it. If the user gives unsuitable input – for example, subtracting more than was previously set – a warning or error message should be shown.

```
(…)
    // Setter Methoden
    public void setMinutes(int minutes) {
        this.minutes = minutes;
    }

    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }
}
```

Users should not have to convert time units from hours and minutes into minutes.

We give the user a method.

```
(…)
    // Setter Methoden
    public void setMinutes(int minutes) {
        this.minutes = minutes;
    }

    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }
}
```

Methods for **every** class:

- ❑ Constructors
- ❑ toString method

More later…

2. Create suitable methods for this class. We should be able to change the time span (set it to any (sensible) value, add and subtract), return it in minutes, and print it. If the user gives unsuitable input – for example, subtracting more than was previously set – a warning or error message should be shown.

```
public class Zeitspanne {
    private int minutes;

    public Zeitspanne() {
        this.minutes = 0;
    }

    public Zeitspanne(int hours, int minutes) {
        this.minutes = minutes + hours * 60;
    }

(…)
}
```

Constructors:

- ❑ Default constructor
- ❑ Explicit constructors (parameterized or parameterless)

```
public class Zeitspanne {
    private int minutes;
(…)
    public Zeitspanne(int hours, int minutes) {
        this.minutes = minutes + hours * 60;
    }

    public Zeitspanne(int minutes) {
        this.minutes = minutes;
    }

    public Zeitspanne(Zeitspanne other) {
        this.minutes = other.minutes;
    }
(…)
}
```

Constructors:

- ❑ Default constructor
- ❑ Explicit constructors (parameterized or parameterless)
- ❑ Copy constructor



Why?

We create an object with another object as a template.

```
public class Zeitspanne {
    private int minutes;
(…)
    public Zeitspanne(int hours, int minutes) {
        this.minutes = minutes + hours * 60;
    }

    public Zeitspanne(int minutes) {
        this.minutes = minutes;
    }

    public Zeitspanne(Zeitspanne other) {
        this.minutes = other.minutes;
    }
(…)
}
```

Constructors:

- ❑ Default constructor
- ❑ Explicit constructors (parameterized or parameterless)
- ❑ Copy constructor

Always exists, unless you write your own constructor.

```
public class Zeitspanne {
    private int minutes;
(…)
    public Zeitspanne(int hours, int minutes) {
        this.minutes = minutes + hours * 60;
    }

    public Zeitspanne(int minutes) {
        this.minutes = minutes;
    }

    public Zeitspanne(Zeitspanne other) {
        this.minutes = other.minutes;
    }
(…)
}
```

Very repetitive…

```java
public class Zeitspanne {
    private int minutes;
    public Zeitspanne(int minutes) {
        this.minutes = minutes;
    }
    public Zeitspanne() {
        this(0);
    }
    public Zeitspanne(int hours, int minutes) {
        this(minutes + hours * 60);
    }
    public Zeitspanne(Zeitspanne other) {
        this(other.minutes);
    }
(…)
}
```

One main parameterized constructor.
All other constructors call this one.

```java
public class Zeitspanne {
    private int minutes;
    public Zeitspanne(int minutes) {
        this.minutes = minutes;
    }
    public Zeitspanne() {
        this(0);
    }
    public Zeitspanne(int hours, int minutes) {
        this(minutes + hours * 60);
    }
    public Zeitspanne(Zeitspanne other) {
        this(other.minutes);
    }
(…)
}
```

```java
public class Zeitspanne {
(…)
    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }

    public String toString() {
        int hours = this.minutes / 60;
        int minutes = this.minutes % 60;

        return hours + ":" + minutes;
    }
}
```

Methods for **every** class:

- ❑ Constructors
- ❑ toString method

```
public class Zeitspanne {
(…)
    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }

    public String toString() {
        int hours = this.minutes / 60;
        int minutes = this.minutes % 60;

        return hours + ":" + minutes;
    }
}
```

We will do these calculations several times…

We write a method!

```
public class Zeitspanne {
(…)
    public int[] toHourMin() {
        int hours = this.minutes / 60;
        int minutes = this.minutes % 60;

        return new int[] {hours, minutes};
    }

    public String toString() {
        int[] split = toHourMin();
        return split[0] + ":" + split[1];
    }

}
```

We will do these calculations several times…

We write a method!

```
public class Zeitspanne {
(…)
    public Zeitspanne(int hours, int minutes) {
        this(minutes + hours * 60);
    }
    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }

}
```

We do these calculations several times…

We write a method!

```
public class Zeitspanne {
(…)
    public static int getMinutes(int hours, int minutes) {
        return hours * 60 + minutes;
    }
    public Zeitspanne(int hours, int minutes) {
        this(minutes + hours * 60);
    }
    public void setTime(int hours, int minutes) {
        this.minutes = hours * 60 + minutes;
    }

}
```

We will do these calculations several times…

We write a method!

```
public class Zeitspanne {
(…)
    public static int getMinutes(int hours, int minutes) {
        return hours * 60 + minutes;
    }
    public Zeitspanne(int hours, int minutes) {
        this(getMinutes(hours, minutes));
    }
    public void setTime(int hours, int minutes) {
        this.minutes = getMinutes(hours, minutes);
    }

}
```

We will do these calculations several times…

We write a method!

```java
public class Zeitspanne {
(…)
    public void addTime(int hours, int minutes) {
        this.minutes += getMinutes(hours, minutes);
    }

    public void subTime(int hours, int minutes) {
        this.minutes -= getMinutes(hours, minutes);
    }
}
```

2. Create suitable methods for this class. We should be able to change the time span (set it to any (sensible) value, add and subtract), return it in minutes, and print it. If the user gives unsuitable input – for example, subtracting more than was previously set – a warning or error message should be shown.

```
public class Zeitspanne {
(…)
    public void addTime(int hours, int minutes) {
        this.minutes += getMinutes(hours, minutes);
    }

    public void subTime(int hours, int minutes) {
        this.minutes -= getMinutes(hours, minutes);
    }
}
```

What if the `minutes` would become negative?

Now: `boolean` return value that says whether the operation was successful.

Later: Exceptions / Errors

```
public class Zeitspanne {
(…)
    public void addTime(int hours, int minutes) {
        this.minutes += getMinutes(hours, minutes);
    }
    public boolean subTime(int hours, int minutes) {
        int minutesToSub = getMinutes(hours, minutes);
        if(this.minutes >= minutesToSub) {
            this.minutes -= minutesToSub;
            return true;
        }
        return false;
    }
}
```

Now: `boolean` return value that says whether the operation was successful.

Later: Exceptions / Errors

2. Create suitable methods for this class. We should be able to change the time span (set it to any (sensible) value, add and subtract), return it in minutes, and print it. If the user gives unsuitable input – for example, subtracting more than was previously set – a warning or error message should be shown.

```
public class Zeitspanne {
(…)
    public boolean isMoreThan(Zeitspanne other) {
        return this.getTimeInMin() > other.getTimeInMin();
    }

}
```

3. Now write a method ‘isMoreThan()’ that takes a second Zeitspanne as an argument and makes it possible to compare two time spans.

StackOverflow Error

```java
public class Factorial {

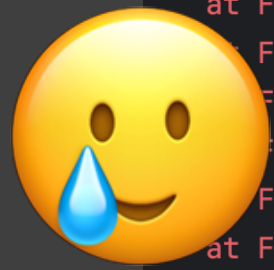
    public static int factorial(int n) {
        return n * factorial(n - 1);
    }

}
```

Scenario: You are supposed to write the method `factorial(int n)`, which should compute $n!$.

1. **You know the formula for $n!$ and implement the method.**

1. **You run the method and see the following:**



```
Exception in thread "main" java.lang.StackOverflowError
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
    at Factorial.factorial(Factorial.java:3)
```

```java
public class Factorial {

    public static int factorial(int n) {
        return n * factorial(n - 1);
    }

}
```

Scenario: You are supposed to write the method `factorial(int n)`, which should compute $n!$.

1. **You know the formula for $n!$ and implement the method.**

1. **You run the method and see the following:**

```
Exception in thread "main"
java.lang.StackOverflowError
```

1. **What do we do now?**

- **Forgot the base case?**

```java
public class Factorial {

    public static int factorial(int n) {
        if(n == 0) {
            return 1;
        }
        return n * factorial(n - 1);
    }
}
```

Scenario: You are supposed to write the method `factorial(int n)`, which should compute $n!$.

1. **You know the formula for $n!$ and implement the method.**

1. **You run the method and see the following:**
   ```
   Exception in thread "main"
   java.lang.StackOverflowError
   ```

1. **What do we do now?**

- **Forgot the base case?** Here yes!
- **Often, however, too many method calls are generated!**
  Then you have to think about how to prevent certain method calls in the recursion.

Kahoot!

Loop Invariants

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Loop Invariants

1. The loop invariant is satisfied **before the loop**.

1. The loop invariant is satisfied **after every execution** of the while body.

1. The loop invariant is satisfied **after the loop**.

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Loop Invariants

1. The loop invariant is satisfied **before the loop**.

1. The loop invariant is satisfied **after every execution** of the while body.

1. The loop invariant is satisfied **after the loop**.

We want to prove the following Hoare triple:

```
{ Precondition }
  while ( Condition ) { Body };
{ Postcondition }
```

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

We want to prove the following Hoare triple:

{ Precondition }
  while (Condition) { Body };
{ Postcondition }

We can do this if an invariant exists for which the following holds:

1. Precondition ⇒ Invariant

1. { Condition ∧ Invariant }
     Body;
   { Invariant } is a valid triple.

3. ¬ Condition ∧ Invariant ⇒ Postcondition

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
…	…	…

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
…	…	…
i	b + i	a - i

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

Derivation Example with Table

Iteration	res	x
0	b	a
1	b + 1	a - 1
2	b + 2	a - 2
3	b + 3	a - 3
…	…	…
i	b + i	a - i

LI: res == b + a -

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

1. Precondition ⇒ Invariant

1. { Condition ∧ Invariant }
   Body;
   { Invariant } is a valid triple.

3. ¬ Condition ∧ Invariant ⇒ Postcondition

LI: res == b + a - x

Is that enough?

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

1. Precondition ⇒ Invariant ✓

1. { Condition ∧ Invariant }
   Body;
   { Invariant } is a valid triple. ✓

3. ¬ Condition ∧ Invariant ⇒ Postcondition ✗

LI: res == b + a - x

Is that enough?

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = a;
    res = b;
    // Loop-Invariante: ??
    while (x > 0) {
      x = x - 1;
      res = res + 1;
    }
    // Postcondition:  res == a + b
    return res;
}
```

1. Precondition ⇒ Invariant ✓

1. { Condition ∧ Invariant }
   Body;
   { Invariant } is a valid triple. ✓

3. ¬ Condition ∧ Invariant ⇒ Postcondition ✓

```
res == b + a – x
&&
x >= 0
```

Example 2

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1
2	n * (n – 1)	n – 2

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1
2	n * (n – 1)	n – 2
3	…	n – 3

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1
2	n * (n – 1)	n – 2
3	…	n – 3
…	…	…

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1
2	n * (n – 1)	n – 2
3	…	n – 3
…	…	…
i	n! / (n – i)!	n – i

```java
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

Derivation Example with Table

Iteration	result	counter
0	1	n
1	n	n – 1
2	n * (n – 1)	n – 2
3	…	n – 3
…	…	…
i	n! / (n – i)!	n – i

LI: result == n! / counter!

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

1. Precondition ⇒ Invariant ✓

1. { Condition ∧ Invariant }
   Body;
   { Invariant } is a valid triple ✓

3. ¬ Condition ∧ Invariant ⇒ Postcondition ✗

LI: result == n!/ counter!

Is that enough?

```
public static int factorial(int n) {
    // Precondition: n >= 0
    int counter;
    int result;

    counter = n;
    result = 1;
    // Loop-Invariante: ??
    while (counter > 0) {
        result = result * counter;
        counter = counter - 1;
    }

    // Postcondition: result = n!
    return result;
}
```

1. Precondition ⇒ Invariant ✓

1. { Condition ∧ Invariant }
   Body;
   { Invariant } is a valid triple ✓

3. ¬ Condition ∧ Invariant ⇒ Postcondition ✓

LI: result == n! / counter!
&& counter >= 0

Recipe for Loop Invariants

There are no perfect recipes!



- Being able to solve loop invariants correctly is a **matter of practice**.
- The following recipe has proven useful for many students in the past.

```java
public int compute(int n, int k){
    // Precondition: n >= 0
    int i = 0;
    int result = 1;

    // Loop-Invariante: ??
    while (i < n) {
        result = result * k;
        i = i + 1;
    }
    // Postcondition result == k^n
    return result;
}
```

1. **Combine loop condition and termination.**

$$\{i <= n\}$$

2. **Combine postcondition and loop body.**

$$\{result == k^n\} \rightarrow \{result == k^i\}$$

3. **Conditions due to methods used or necessary preconditions.***

$$\{n \geq 0\}$$

```
{ i <= n
  && result == k^i
  && n >= 0}
```

* Not necessary in all cases in EProg.

Further Material

Videos about Loop Invariants, Hoare Triples, Weakest Precondition, Exam Prep etc

https://eprog23.wixstudio.com/hs23

Preview

Exercise 2: Loop Invariant

1. The precondition and postcondition are given for the following program.

```
public int compute(int n) {
    // Precondition:  n >= 0
    int x;
    int res;

     x = 0;
    res = x;

    // Loop Invariant:
    while (x <= n) {
        res = res + x;
        x = x + 1;
    }
    // Postcondition:  res == ((n + 1) * n) / 2
    return res;
}
```

Write the loop invariant in the file “LoopInvariante.txt”.

Exercise 2: Loop Invariant

2. The precondition and postcondition are given for the following program.

```
public int compute(int a, int b) {
    // Precondition:  a >= 0
    int x;
    int res;

    x = 0;
    res = b;

    // Loop Invariant:
    while (x < a) {
        res = res - 1;
        x = x + 1;
    }
    // Postcondition:  res == b - a
    return res;
}
```

Write the loop invariant in the file “LoopInvariante.txt”.

Exercise 4: Minesweeper (Bonus)

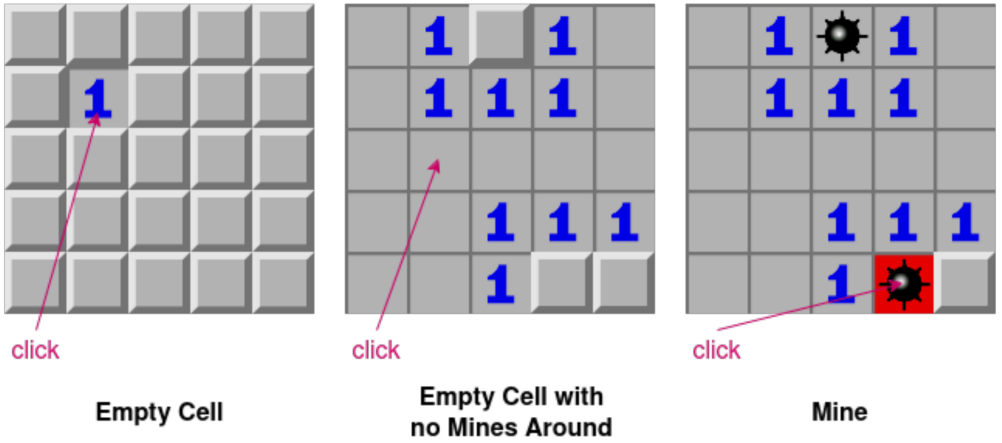


Figure 1: Game boards after the first, second, and third click from left to right.

Attention: This task gives bonus points (see "Leistungskontrolle" at `www.vvz.ethz.ch`). The task must be solved independently and alone. Submission is done as usual by pushing to your Git repository on the ETH server. The last push before the submission deadline is binding. Even if you commit before the deadline but push after the deadline, this counts as a late submission. Please also read the general rules.

Thank you very much for your attention

Have a great week